UNIVERSIDAD NACIONAL DE SAN MARTÍN · BASES DE DATOS

BairesGuessr

Juego de geolocalización cultural en CABA con persistencia políglota

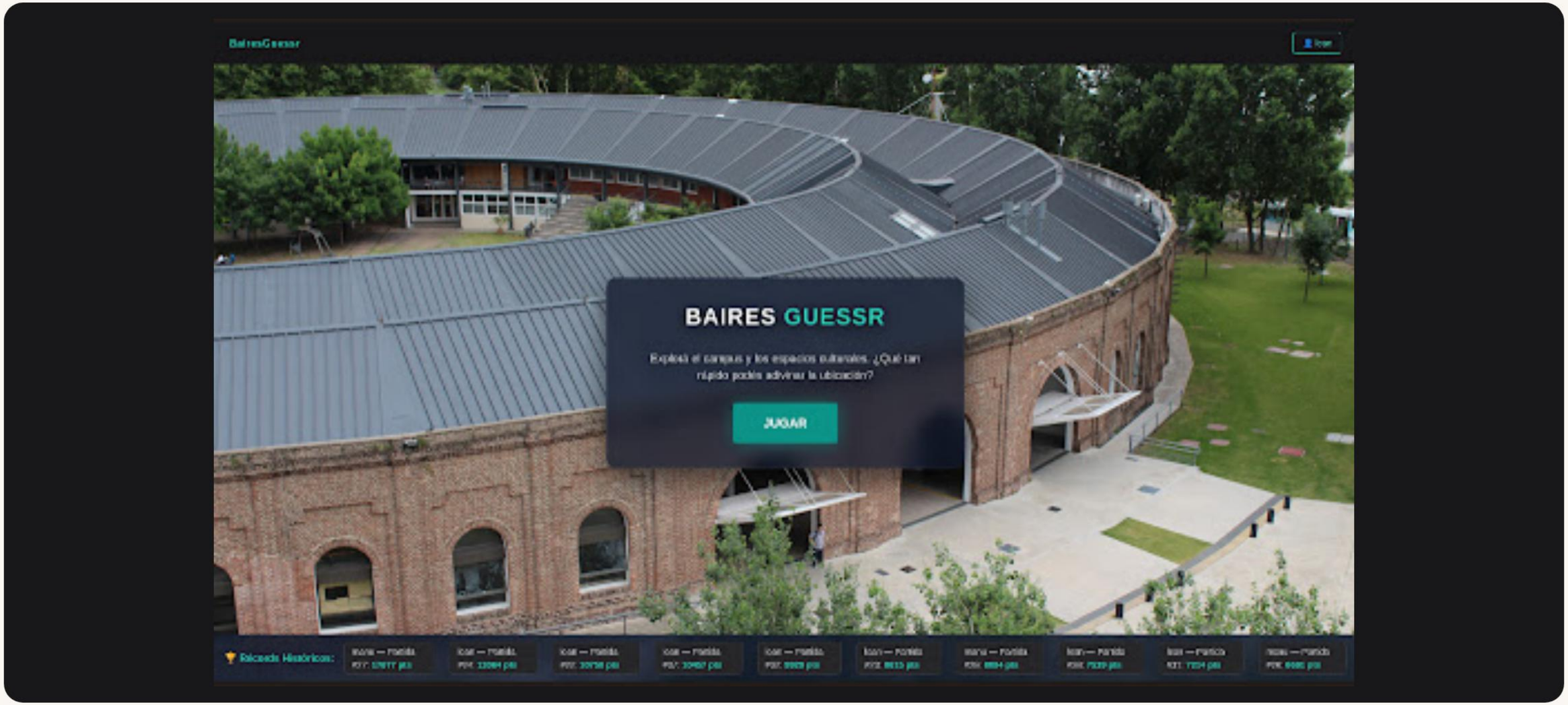
Chavez · Avila · Giuggiolini · Millauro · Mastrangelo

Grupo 5 · Tecnicatura en Programación Informática

Docentes: Rodrigo Casco · Carlos Gabriel Flores

01 · CONTEXTO Y DOMINIO

Problema Seleccionado



02 · ORIGEN DE LOS DATOS

Dataset Abierto del GCBA

3.053
Registros culturales originales

30 → 6
Columnas conservadas tras depuración

791
Muestras seleccionadas para el juego

Atributos Esenciales Conservados

`fid`, `ESTABLECIMIENTO`, `FUNCION_PRINCIPAL`, `BARRIO`, `LATITUD` y `LONGITUD`. Se descartaron teléfonos, redes y aforos.

Categorías Elegidas (791 lugares)

Salas de Teatro (311), Monumentos y Lugares Históricos (304) y Museos (176) por su alta reconocibilidad visual.

02 · CURADURÍA Y ENRIQUECIMIENTO

Limpieza con Pandas y Búsqueda en Mapillary

filtrarcsv.py · Depuración de columnas y categorías

```
import pandas as pd

df = pd.read_csv('espacios-culturales.csv')
cols = ['fid', 'ESTABLECIMIENTO', 'FUNCION_PRINCIPAL',
        'BARRIO', 'LATITUD', 'LONGITUD']
cats = ['SALA DE TEATRO', 'MUSEO',
        'MONUMENTOS Y LUGARES HISTORICOS']

df_final = df[cols][df['FUNCION_PRINCIPAL'].isin(cats)]
df_final.to_csv('espacios-culturales-limpio.csv',
index=False)
```

buscar_mapillary_id() · Entorno y validación visual

```
def buscar_mapillary_id(session, lat: float, lng: float) → str |
None:
    for delta in (0.0005, 0.0012):
        bbox = f"{lng - delta},{lat - delta},{lng + delta},{lat +
delta}"
        params = {"access_token": MAPILLARY_TOKEN, "fields": "id",
                  "bbox": bbox, "limit": 1}
        try:
            resp = session.get("https://graph.mapillary.com/images",
                               params=params, timeout=5)
            if resp.status_code == 200 and resp.json().get("data"):
                return str(resp.json()["data"][0]["id"])
        except requests.RequestException as e:
            print(f"Error consultando Mapillary: {e}")
            break
    return None
```

Filtrado inicial: Reduce el dataset crudo a 791 espacios reconocibles.

Cobertura 360°: Busca fotos en radio de ~50m a ~130m y descarta lugares sin vista.

Anomalías del Esquema Universal Inicial

Redundancia de Datos

Email, clave y estado de partida repetidos en cada jugada.

Anomalía de Actualización

Modificar credenciales exige actualizar múltiples filas históricas.

Anomalía de Inserción

Imposible dar de alta un usuario sin partidas previas.

Anomalía de Eliminación

Borrar rondas elimina accidentalmente al jugador del sistema.

03 · PROCESO DE NORMALIZACIÓN (2/2)

Evolución hacia Tercera Forma Normal (3FN)

PASO 01 · 1FN

Valores Atómicos

Coordenadas separadas en columnas numéricas individuales sin listas.

PASO 02 · 2FN

Sin Dependencias Parciales

Descomposición de clave compuesta separando PARTIDA de RONDA.

PASO 03 · 3FN

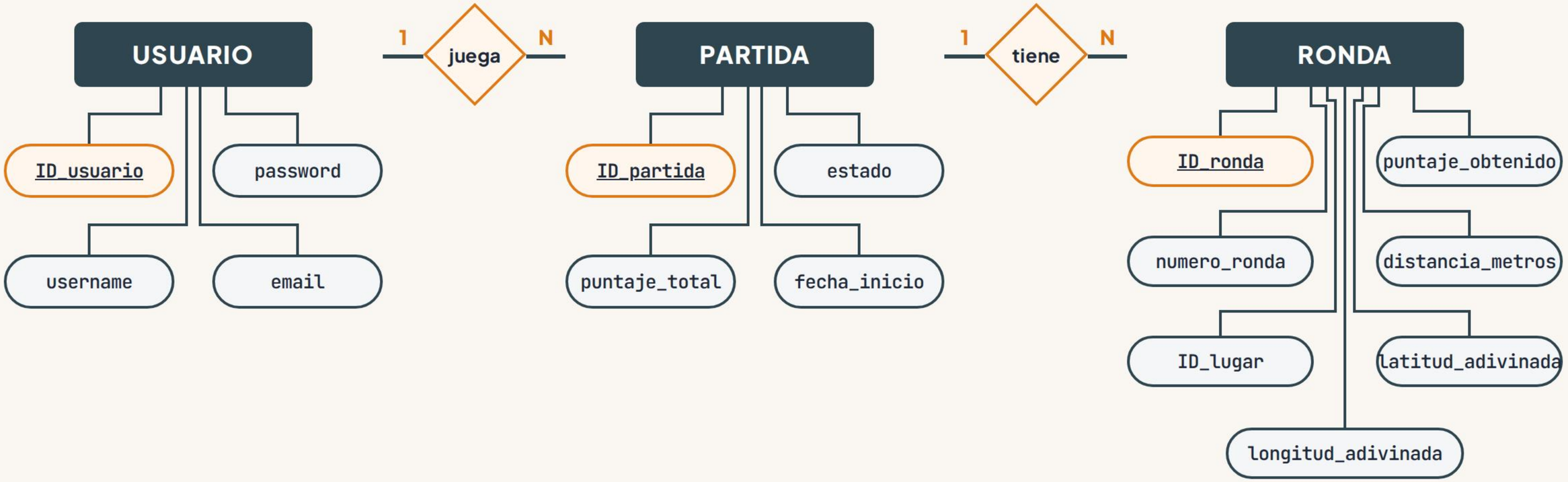
Sin Dependencias Transitivas

Extracción de USUARIO y referencia externa mediante id_lugar.

04 · MODELADO CONCEPTUAL

Diagrama Entidad–Relación (DER)

Notación Chen · Relaciones 1:N · Atributos clave subrayados



Transformación a Tablas PostgreSQL

🔑 Primary Key (PK) · 🗝️ Foreign Key (FK)

USUARIO	
🔑 **id_usuario**	SERIAL PK
username	VARCHAR(50)
email	VARCHAR(100)
password	VARCHAR(255)

PARTIDA	
🔑 **id_partida**	SERIAL PK
🗝️ **id_usuario**	INT FK
puntaje_total	INT
fecha_inicio	TIMESTAMP
estado	VARCHAR(20)

RONDA	
🔑 **id_ronda**	SERIAL PK
🗝️ **id_partida**	INT FK
numero_ronda	INT
id_lugar	VARCHAR(50)
latitud_adivinada	DECIMAL(10,8)
longitud_adivinada	DECIMAL(11,8)
distancia_metros	DECIMAL(10,2)
puntaje_obtenido	INT

Script DDL: Creación de Tablas y Restricciones

PK · FK (ON DELETE RESTRICT) · CHECK de dominio

```
-- 1. Tabla independiente: USUARIO
CREATE TABLE USUARIO (
    id_usuario SERIAL PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL
);

-- 2. Tabla dependiente de USUARIO: PARTIDA
CREATE TABLE PARTIDA (
    id_partida SERIAL PRIMARY KEY,
    id_usuario INT NOT NULL,
    puntaje_total INT NOT NULL DEFAULT 0,
    fecha_inicio TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP,
    estado VARCHAR(20) NOT NULL,
    CONSTRAINT fk_partida_usuario
        FOREIGN KEY (id_usuario) REFERENCES USUARIO
(id_usuario)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT chk_puntaje_total_positivo CHECK
(puntaje_total ⩾ 0)
);
```

```
-- 3. Tabla dependiente de PARTIDA: RONDA
CREATE TABLE RONDA (
    id_ronda SERIAL PRIMARY KEY,
    id_partida INT NOT NULL,
    numero_ronda INT NOT NULL,
    id_lugar VARCHAR(50) NOT NULL,
    latitud_adivinada DECIMAL(10, 8) NOT NULL,
    longitud_adivinada DECIMAL(11, 8) NOT NULL,
    distancia_metros DECIMAL(10, 2) NOT NULL,
    puntaje_obtenido INT NOT NULL,
    CONSTRAINT fk_ronda_partida
        FOREIGN KEY (id_partida) REFERENCES PARTIDA (id_partida)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT chk_latitud_adivinada_rango
        CHECK (latitud_adivinada BETWEEN -90.0 AND 90.0),
    CONSTRAINT chk_longitud_adivinada_rango
        CHECK (longitud_adivinada BETWEEN -180.0 AND 180.0),
    CONSTRAINT chk_numero_ronda_valido CHECK (numero_ronda > 0),
    CONSTRAINT chk_distancia_positiva CHECK (distancia_metros ⩾
0.0),
    CONSTRAINT chk_puntaje_obtenido_positivo CHECK (puntaje_obtenido
⩾ 0)
);
```

06 · MODELO NOSQL (1/3)

Estructura Documental en MongoDB

bairesguessr · espacios_culturales

```
{
  "_id": ObjectId("652f1a9b8c3d2e1f0a4b5c6d"),
  "fid": 123,
  "establecimiento": "El Ateneo Grand Splendid",
  "funcion_principal": "BIBLIOTECA",
  "barrio": "RECOLETA",
  "ubicacion": {
    "type": "Point",
    "coordinates": [-58.3924, -34.5960]
  },
  "mapillary_id": "498271635012948"
}
```

Documento autocontenido: Metadatos, foto 360° y geometría en un único objeto.

Flujo nativo JSON: Serialización directa entre MongoDB, Kotlin y React.

0 JOINs: Recupera toda la información del lugar en una sola lectura.

Consultas espaciales nativas: Búsquedas geográficas resueltas directo en el motor.

Script en Python para Poblar MongoDB

PyMongo · ThreadPoolExecutor · Índice 2dsphere

Armado del documento GeoJSON

```
def procesar_fila(row: dict) → dict | None:
    lat, lng = float(row["LATITUD"]), float(row["LONGITUD"])
    with requests.Session() as session:
        mapillary_id = buscar_mapillary_id(session, lat, lng)
    if not mapillary_id:
        return None
    return {
        "fid": int(row["fid"]),
        "establecimiento": row["ESTABLECIMIENTO"],
        "funcion_principal": row["FUNCION_PRINCIPAL"],
        "barrio": row["BARRIO"],
        "ubicacion": {"type": "Point", "coordinates": [lng,
lat]},
        "mapillary_id": mapillary_id,
    }
```

Carga masiva paralela e indexación espacial

```
def importar_datos():
    client = MongoClient("mongodb://localhost:27017/")
    collection = client["bairesguessr"]["espacios_culturales"]
    collection.delete_many({})

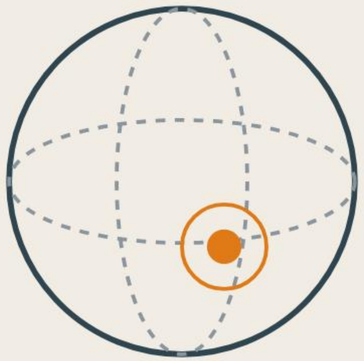
    with open("espacios-culturales-limpio.csv", encoding="utf-8") as
f:
        filas = list(csv.DictReader(f))

    with ThreadPoolExecutor(max_workers=10) as executor:
        docs = [d for d in executor.map(procesar_fila, filas) if d]

    if docs:
        collection.insert_many(docs)
        collection.create_index([("ubicacion", "2dsphere")])
```

06 · MODELO NOSQL (3/3)

Estándar GeoJSON e Índice 2dsphere



```
createIndex({ ubicacion: "2dsphere" })
```

⭐ **Motivo clave de elección:** El soporte nativo JSON/BSON habilita estas tecnologías sin conversiones.

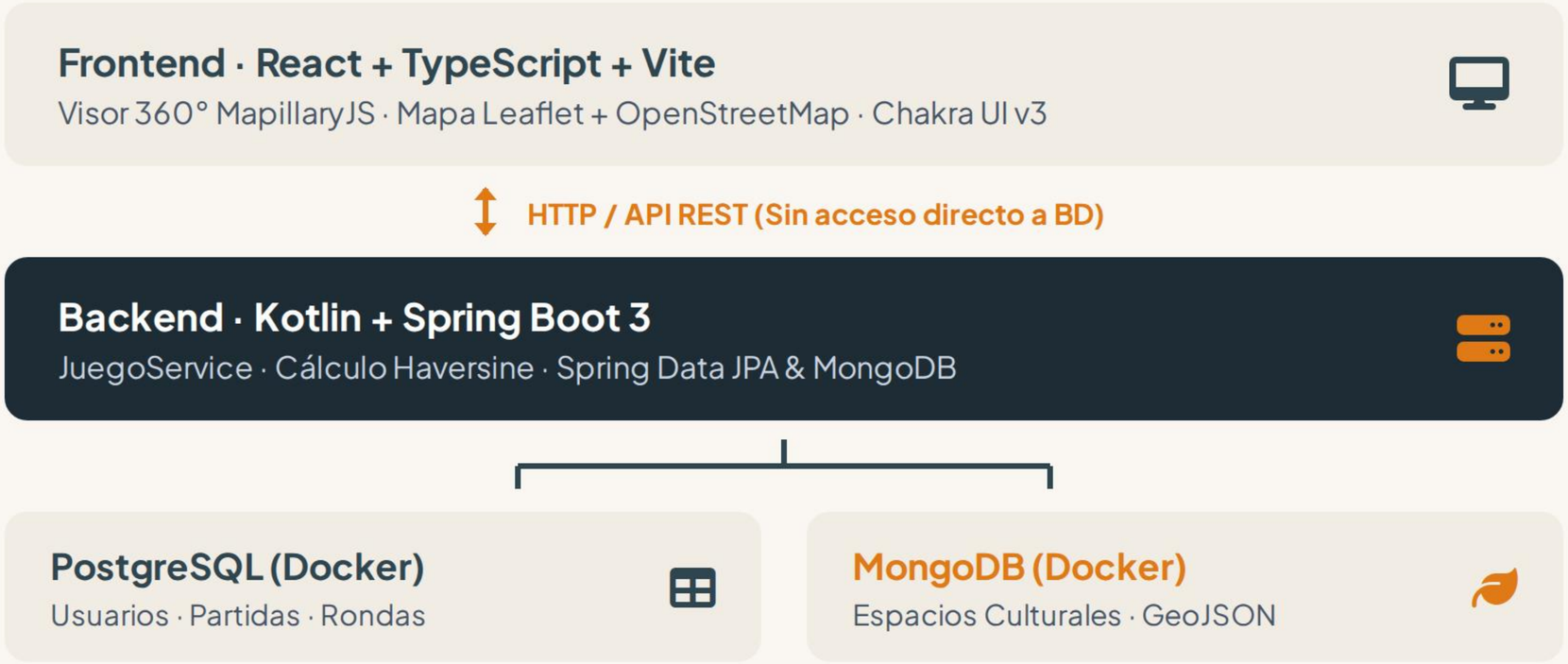
📍 **Estándar GeoJSON Point:** Arreglo ordenado `[longitud, latitud]` interoperable con Leaflet.

🌐 **Índice 2dsphere:** Calcula distancias sobre la curvatura terrestre en lugar de planos cartesianos.

🧭 **Consultas de proximidad:** Soporta búsquedas por radio y selección aleatoria veloz.

07 · DISEÑO DEL SISTEMA

Arquitectura General de la Aplicación



08 · COMUNICACIÓN DUAL EN JUEGOSERVICE

Vínculo entre PostgreSQL y MongoDB

idLugar · Referencia cruzada sin clave foránea nativa

1. generarNuevaRonda() · MongoDB → PostgreSQL

```
// Obtiene un lugar inédito desde MongoDB
val todosLosLugares = espacioCulturalRepository.findAll()
val seleccionado = disponibles.random()
val idMongo = seleccionado.id!!

// Guarda su ObjectId como texto en PostgreSQL
val nuevaRonda = Ronda(
    partida = partida,
    numeroRonda = rondasExistentes.size + 1,
    idLugar = idMongo
)
val guardada = rondaRepository.save(nuevaRonda)
```

Al iniciar la ronda, el backend toma el `_id` del documento en MongoDB y lo persiste como cadena de texto en la columna `id_lugar` de la tabla `RONDA` en PostgreSQL.

Registro en SQL: Guarda el `_id` documental en la columna `id_lugar`.

2. procesarAdivinanza() · PostgreSQL → MongoDB

```
// Recupera la ronda activa desde PostgreSQL
val ronda = rondaRepository.findById(idRonda).orElseThrow()

// Busca ese mismo ObjectId en MongoDB
val espacio = espacioCulturalRepository
    .findById(ronda.idLugar).orElseThrow()

val latReal = espacio.ubicacion!!.latitud
val lonReal = espacio.ubicacion!!.longitud
```

Al responder o pasar de ronda, lee `id_lugar` desde PostgreSQL y consulta ese mismo ID en MongoDB para obtener las coordenadas reales y excluir lugares ya jugados.

Cruce en memoria: Obtiene coordenadas reales para calcular distancia con Haversine.

09 · CAPA DE ACCESO A DATOS

Repositorios: MongoDB y PostgreSQL

Spring Data MongoDB · Spring Data JPA

MongoDB · EspacioCulturalRepository.kt

```
@Repository
interface EspacioCulturalRepository :
    MongoRepository {

  @Aggregation(pipeline = ["{ '\$sample': { 'size': 1 } }"])
  fun obtenerAleatorio(): EspacioCultural?

  @Aggregation(pipeline = [
    "{ '\$match': { '_id': { '\$nin': ?0 } } }",
    "{ '\$sample': { 'size': 1 } }"
  ])
  fun obtenerAleatorioExcluyendo(
      idsExcluidos: List): EspacioCultural?
}
```

PostgreSQL · Repositories JPA

```
interface PartidaRepository : JpaRepository {
  @Query("""SELECT p FROM Partida p JOIN FETCH p.usuario
            WHERE p.estado = 'FINALIZADA'
            ORDER BY p.puntajeTotal DESC LIMIT 10""")
  fun findTop10Finalizadas(): List

  fun findByUsuarioIdOrderByFechaInicioDesc(id: Int): List
}

interface RondaRepository : JpaRepository {
  fun findByPartidaIdOrderByNumeroRondaAsc(id: Int): List
}

interface UsuarioRepository : JpaRepository {
  fun existsByUsername(username: String): Boolean
}
```

Pipelines BSON: Selección aleatoria y exclusión de lugares ya jugados en motor.

Consultas relacionales: JPQL con `JOIN FETCH` para récords e historial.

10 · CONSULTAS REPRESENTATIVAS

Historial de Partidas y Tabla de Récords

Consultas relacionales en PostgreSQL renderizadas en React

Historial por Usuario

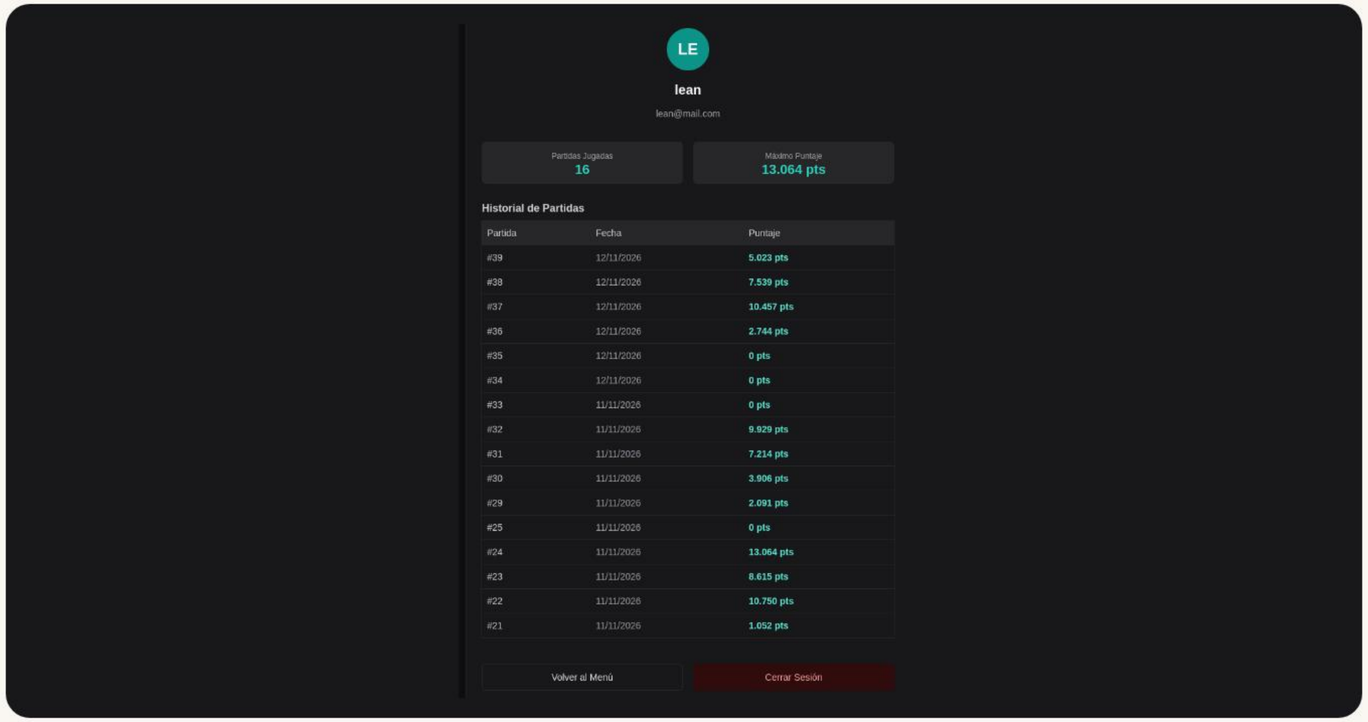
`GET /api/partidas/usuario/{id}`

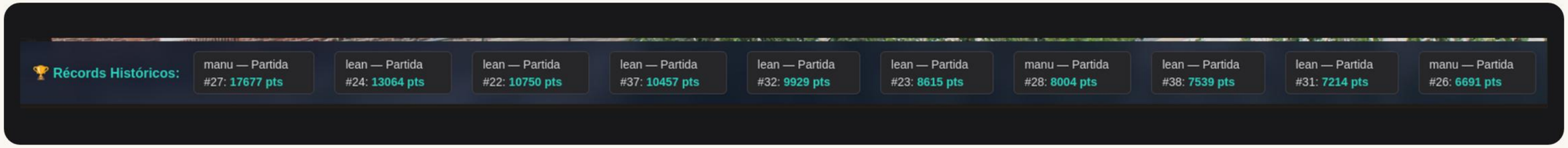
Lista partidas disputadas, fecha y puntaje total del jugador.

Top 10 Récords Globales

`GET /api/partidas/leaderboard`

Ranking histórico ordenado descendentemente por puntaje final.





Problemas Encontrados y Decisiones Tomadas

Reasignación Post-Checkpoint

Separación definitiva de partidas en SQL y catálogo en NoSQL tras revisar el alcance inicial.

Despliegue Docker Compose

Inicialización automática de tablas, datos e índices espaciales sin pasos manuales entre sistemas operativos.

docker-compose.yml · Persistencia Políglota Automatizada

```
version: '3.8'
services:
  postgres:
    image: postgres:16-alpine
    container_name: bairesguessr_postgres
    restart: always
    environment:
      POSTGRES_DB: bairesguessr
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgrespassword
    ports:
      - "5432:5432"
    volumes:
      - ./database/script.sql:/docker-entrypoint-initdb.d/script.sql
      - ./database/data:/var/lib/postgresql/data
  mongo:
    image: mongo:7.0
    container_name: bairesguessr_mongo
    restart: always
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db
      - ./database/scripts_nosql/espacios_culturales.json:/docker-entrypoint-
initdb.d/espacios_culturales.json:ro
      - ./database/scripts_nosql/init-mongo.js:/docker-entrypoint-
initdb.d/init-mongo.js:ro
volumes:
  mongo_data:
```



12 · APLICACIÓN FUNCIONANDO

Vamos a jugar una partida

5 rondas · Entorno 360° Mapillary · PostgreSQL + MongoDB en vivo

13 · BALANCE DEL PROYECTO

Conclusiones Principales

Caso de Uso por Motor

Ningún motor reemplaza universalmente a otro: cada paradigma resuelve mejor su dominio.

Docker y Orden del Repo

Contenedores reproducibles y carpetas organizadas donde cada componente cumple una función.

Tratamiento de los Datos

La limpieza previa con scripts y la normalización garantizan calidad e integridad.

BAIRESGUESSR · GRUPO 5

FIN

¡Muchas gracias por su atención!

Espacio abierto para preguntas y consultas

Chavez · Avila · Giuggiolini · Millauro · Mastrangelo